

Лабораторна робота №6

Тема: Основи роботи з Git. Робота з гілками: командна розробка

Метою даної є отримання базових навичок командної роботи в системі контролю версій git. Робота в GitHub.

Задачі:

- розділити основну задачу на невеликі підзадачі за допомогою системи Trello;
- розподілити відповідальність між учасниками групи;
- створити репозиторій проекту для 4-х облікових записів.

Теоретична частина

Майже кожна система контролю версій підтримує гілки (branches) в певній мірі. Галуження - це відмежування від основної лінії розробки для продовження своєї частини роботи та уникнення конфліктів з основною лінією. Коли ви фіксуєте зміни, Git зберігає об'єкт фіксації, що містить вказівник на знімок змісту, який ви додали. Цей об'єкт також містить ім'я та поштову адресу автора, набране вами повідомлення та вказівники на фіксацію або фіксації, що були прямо до цієї фіксації (батько чи батьки): нуль для першої фіксації, одна фіксація для нормальної фіксації, та декілька фіксацій для фіксацій, що вони є результатом злиття двох чи більше гілок.

Коли ви створили фіксацію за допомогою `git commit`, Git обчислив контрольну суму кожної теки та зберігає ці об'єкти дерева в сховищі Git. Потім Git створює об'єкт фіксації, що зберігає метадані та вказівник на корінь дерева проекту, щоб він міг відтворити цей знімок, коли потрібно.

Якщо ви зробите якісь зміни та зафіксуєте знову, наступна фіксація буде зберігати вказівник на попередню.

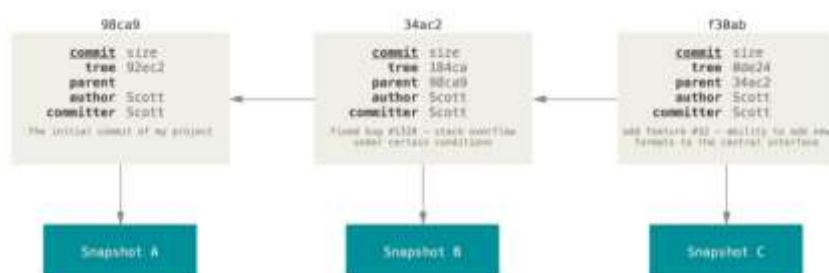


Рисунок 5.1 – Фіксації а їх предки

Гілка в Git це просто легкий вказівник, що може пересуватись, на одну з цих фіксацій. Коли ви почнете робити фіксації, вам надається гілка `master`, що вказує на останню зроблену фіксацію. Щоразу ви фіксуєте, вона переміщується вперед автоматично.

Створення нової гілки створює новий вказівник, щоб ви могли пересуватися. Це робиться за допомогою команди `git branch`.

Особливий вказівник під назвою `HEAD` вказує на локальну гілку, на якій ви знаходитесь.

Щоб переключитися на існуючу гілку, треба виконати команду `git checkout`.

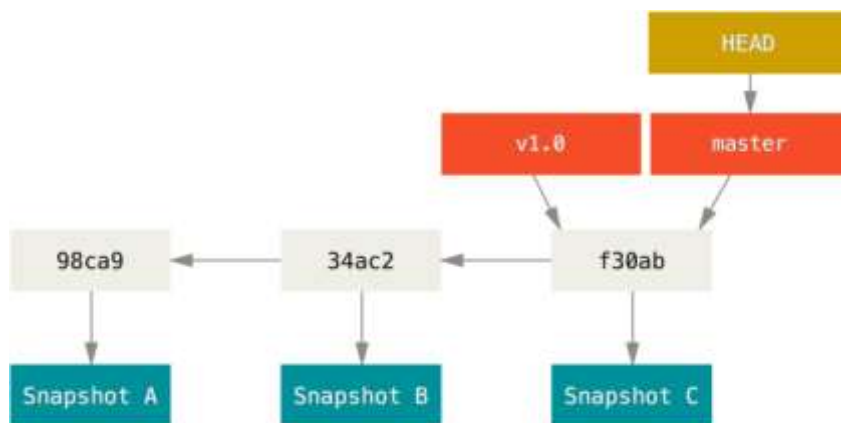


Рисунок 5.2 – Гілка та її історія фіксацій

Важливо зауважити, що коли ви переключаєте гілки в Git, файли у вашій робочій теці змінюються. Якщо ви переключаєтесь до старшої гілки, ваша робоча тека буде повернута до того стану, який був на момент останнього фіксування у тій гілці. Якщо Git не може зробити це без проблем, він не дасть вам переключитися взагалі.

Оскільки гілки в Git це насправді простий файл, що містить 50 символів контрольної суми SHA-1 фіксації, на яку вказує, гілки дешево створювати та знищувати. Створити гілку так же швидко, як записати 41 байт до файлу(40 символів та символ нового рядку).

Додавання розробників на GitHub:

Для того, щоб додати розробника до роботи над репозиторієм, перейдіть до розділу `Settings` і оберіть `Collaborators`:

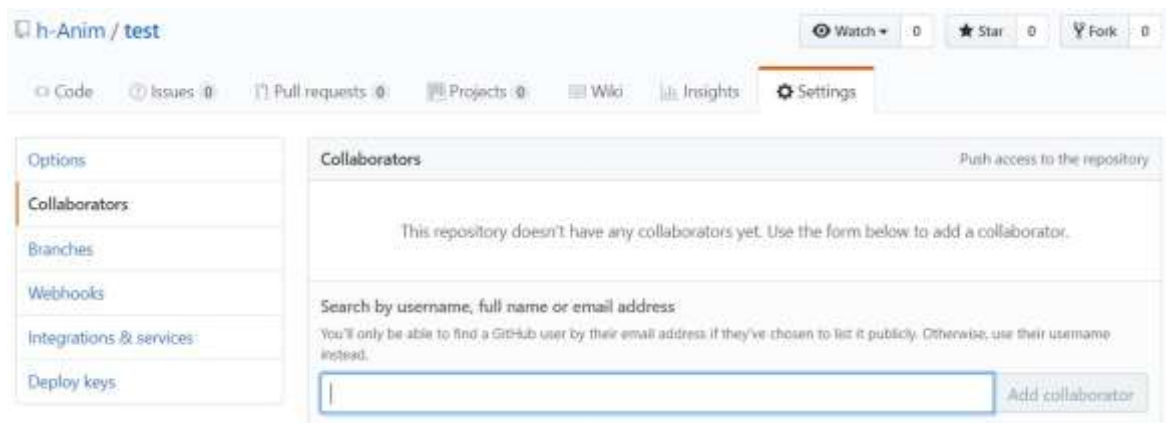


Рисунок 5.3 – Додавання учасників до проекту

У полі введення, що з'явилося, ви можете додати користувача за його ім'ям на GitHub, або адресою електронної пошти. Після чого, доданий вами користувач може клонувати і синхронізувати загальний репозиторій, а також здійснювати commit безпосередньо в загальний репозиторій.

Отримання змін

Для отримання змін з віддаленого репозиторію, використовується команда git fetch:

```
MINGW64:/e/Docs/method/TRPO/src/helloworld
Anim@Anim-PC MINGW64 /e/Docs/method/TRPO/src/helloworld (master)
$ git fetch

Anim@Anim-PC MINGW64 /e/Docs/method/TRPO/src/helloworld (master)
$ git fetch origin

Anim@Anim-PC MINGW64 /e/Docs/method/TRPO/src/helloworld (master)
$ git remote -v
origin https://github.com/h-Anim/helloworld (fetch)
origin https://github.com/h-Anim/helloworld (push)

Anim@Anim-PC MINGW64 /e/Docs/method/TRPO/src/helloworld (master)
$
```

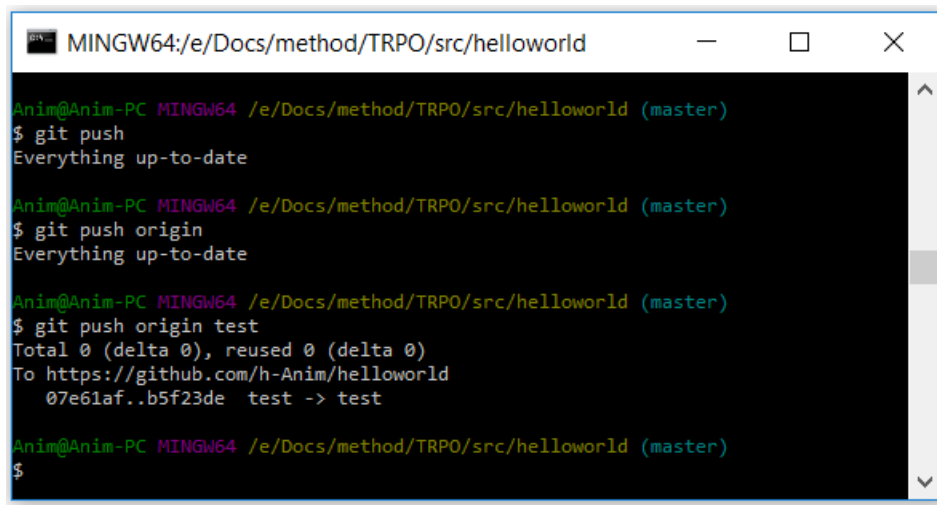
Рисунок 5.4 – Вікно після запуску команди git fetch

Команда може викликатися як без параметрів, так і з зазначенням репозиторію, з яким відбувається синхронізація.

Для того, щоб отримати зміни з віддаленого репозиторію і застосувати їх до поточної гілки, використовується команда git pull:

Відправлення змін:

Для того, щоб відправити локальні зміни до віддаленого репозиторію, використовується команда `git push`:



```
MINGW64:/e/Docs/method/TRPO/src/helloworld

Anim@Anim-PC MINGW64 /e/Docs/method/TRPO/src/helloworld (master)
$ git push
Everything up-to-date

Anim@Anim-PC MINGW64 /e/Docs/method/TRPO/src/helloworld (master)
$ git push origin
Everything up-to-date

Anim@Anim-PC MINGW64 /e/Docs/method/TRPO/src/helloworld (master)
$ git push origin test
Total 0 (delta 0), reused 0 (delta 0)
To https://github.com/h-Anim/helloworld
    07e61af..b5f23de  test -> test

Anim@Anim-PC MINGW64 /e/Docs/method/TRPO/src/helloworld (master)
$
```

Рисунок 5.5 – Вікно після запуску команди `git push`

У першому випадку, зміни будуть відправлені до поточного встановленого віддаленого репозиторію для поточної гілки. У другому випадку зміни будуть відправлені в `origin`. У третьому випадку, зміни гілки `test` будуть відправлені в `origin`.

Хід роботи

1. Створити репозиторій на GitHub.
2. Сформулювати команду із 4-х осіб, завести облікові записи в системі GitHub.com. Вибрати людину, яка буде координатором проекту з повноваженнями адміністратора репозиторію проекту та створити цей репозиторій. Кожен із учасників команди повинен мати доступ до репозиторію для того, щоб мати можливість отримувати код проекту. Координатор проекту має доступ і зміну коду. Крім загального репозиторію кожен із учасників команди повинен мати свій репозиторій у якому зберігатимуться версії коду, з яким він працював.
3. Для кожної команди розподілити завдання між учасниками. Кожен член команди виконує свою частину коду (поділ робить керівник команди або домовляються в Trello).
4. Створити локальний репозиторій.

Приклад задачі. Дано одновимірний масив X, який складається з 12 елементів. Замінити одиницями всі парні елементи масиву, які знаходяться після найменшого елемента. Вивести на друк заданий і новоутворений масиви.

Наприклад, вхідний масив $x=\{5,2,3,4,5,6,-7,8,9,-1,2,3\}$, тоді новоутворений буде $(5\ 2\ 3\ 4\ 5\ 6\ -7\ 1\ 9\ -1\ 1\ 3)$

На іншій гілці розглянути, наприклад, такий вх. масив $x=\{-1,2,3,4,5,6,7,8,9,-1,2,3\}$

Для пошуку мінімального елемента в циклі застосовують умову :

if min<=x[i] {min=x[i]; i_min=i};

Приклад розбиття задачі на підзадачі:

- 1) підключення бібліотек, створення шаблону структури програми, опис змінних, Ініціалізація масиву x.
- 2) виведення масиву (розмір n)
- 3) пошук мінімального елемента, індекс (min, i_min);
- 4) замінити 1 всі парні після i_min;
5. Створити в локальному репозиторії програму, яка буде вирішувати поставлену задачу згідно завдання.
6. Синхронізувати локальний репозиторій з створеним на GitHub.
7. В першій версії проекту зробити ініціалізацію масиву в коді програми.
8. Передбачити, можливість, що послідовність може складатись з елементів, які не будуть задовольняти умову задачі. Для такого випадку створити нову гілку з новими значеннями послідовності, заданими в коді.
9. Кожному учасникові створити гілку, яка дозволить реалізувати один з наступних пунктів:
 - a) вести значення елементів масиву та його розмір з клавіатури;
 - b) заповнити вхідний масив за допомогою генератора випадкових чисел, розмір масиву ввести з клавіатури;
 - c) організувати вивід елементів масиву в рядок з використанням форматowanego виводу;
 - d) організувати вивід елементів масиву з використанням циклу While.
10. Якщо учасник вносив зміни до файлу, то фіксувати всі зміни, документувати їх. Організувати перегляд історії змін.
11. Синхронізувати локальний репозиторій з репозиторієм на GitHub.

Примітка: Якщо, хтось з учасників не справляється або в результаті їхньої роботи виникають помилки, то члени команди можуть внести зміни, при бажанні можна створити власну гілку. Зміни зберегти.

Вимоги і рекомендації. Назва проекту і всі коментарі до нього повинні бути змістовними. Це необхідно для того, щоб розуміти суть проекту і змін, а також відслідковувати їх. Крім того, допоможе людині, яка бачить проект вперше, розібратись в ньому.

ЗАВДАННЯ

Варіант 1

Дано цілочисловий масив А, який складається з 12 елементів. Створити масив С, який складається з елементів масиву А, але починаючи з другого непарного елемента, поділити всі решту елементів масиву А на цей непарний елемент. Вивести на друк заданий і новоутворений масиви. (Передбачити випадок, що таких елементів може не бути, або їх буде < 2 , пункт 8).

Варіант 2

В заданому масиві А, який містить 14 елементів, замінити всі від'ємні елементи, що знаходяться на парних місцях, середнім арифметичним значенням, обчисленим із всіх від'ємних елементів; елементи, що знаходяться на непарних місцях, замінити середнім значенням всіх додатних елементів масиву (передбачити випадок, що всі елементи або від'ємні (замінити на 0), або додатні (замінити на 1), пункт 8). Заданий і перетворений масиви вивести на друк.

Варіант 3

Дано цілочисловий масив А, який складається з 12 елементів. Створити масив С, який складається з остач ділення елементів масиву А на ціле число k ($k < 10$), яке вводиться з клавіатури (передбачити, що k може бути $= 0$, пункт 8). Обчислити добуток ненульових елементів масиву С. Вивести на друк заданий і новоутворений масиви.

Варіант 4

В заданому масиві В, який містить 14 елемент, замінити всі від'ємні елементи, що знаходяться на парних місцях, максимальним його елементом, а на

непарних місцях – максимальним серед від’ємних елементів масиву (передбачити випадок, що масиві тільки додатні елементи, пункт 8). Заданий і новоутворений масиви вивести на друк.

Основні команди для роботи з гілками у Git:

`git branch` – вивід списку існуючих гілок;

`git branch -v` – вивід списку існуючих гілок та їх останніх коммітів;

`git branch -vv` – вивід списку існуючих гілок, що відстежуються;

`git branch --merged` – вивід списку гілок, що злиті із поточною;

`git branch --no-merged` – вивід списку гілок, що не злиті із поточною;

`git branch -u [remote]/[remotebranch]` – прив’язка поточної локальної гілки до віддаленої;

`git branch [branchname]` – створення гілки;

`git branch -d [branchname]` – видалення гілки;

`git branch -D [branchname]` – форсоване видалення гілки;

`git checkout [branchname]` – переключення на гілку;

`git checkout -b [branchname]` – створення гілки та переключення на неї;

`git checkout -b [branchname] [remote]/[remotebranch]` – створення гілки, що відстежує віддалену гілку(з можливістю зміни імені локальної гілки);

`git checkout --track [remote]/[remotebranch]` – створення гілки, що відстежує віддалену гілку;

`git merge [branchname]` – злиття заданої гілки з поточною;

`git push [remote] [localbranch]` – відправлення локальної гілки на сервер;

`git push --delete [remotebranch]` – видалення гілки з сервера;

`git rebase [branch]` – перебазування поточної гілки на задану;

`git rebase [basebranch] [topicbranch]` – перебазування гілок;

`git pull --rebase` – автоматичне перебазування при злитті.

Посилання на джерела

1. <https://plug-community.gitbook.io/plug-c-qt-roadmap-book/naivazhlivishii-rozdil-u-cii-knizhci/bazova-robota-z-git-ta-stvorenniya-vlasnogo-repozitoriyu/stvoryuyemo-repozitorii-ta-zalivayemo-na-github> (на укр. мові)
2. <https://git-scm.com/book/ru/v2/> (на рос. мові)
3. <https://try.github.io/levels/1/challenges/1> (на англ. мові)